Connecting the Dots

From Hilbert's 17th Problem to a Python bug finder

RISE

The Prompt That Started Everything

> ``Produce mathematics at the level of
>
> **Vladimir Voevodsky** (Fields Medal, foundations of mathematics)
>
> but directed toward something the legendary
>
> **Nikolaj Bjørner** (co-creator of Z3) could actually use."

Voevodsky		**Bjørner**
Fields Medal 2002		Co-creator of Z3
Homotopy type theory	$\times$	SMT solving at scale
Proof = computation		Proof = solver output
Foundation-shaking math		*Industrial verification*

The AI generated dozens of ideas. Most were slop.

One thread survived.

Five "Random" Dots

1900
Hilbert's
17th Problem

1974
Stengle's
Positivstellensatz

2000
Parrilo's SOS
(control theory)

2004
Prajna's barriers
(ctrl $\rightarrow$ verif.)

20 techniques
we stole

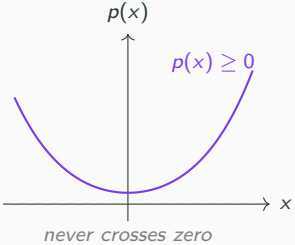
Pure math $\rightarrow$ real algebraic geometry $\rightarrow$ control theory $\rightarrow$ verification $\rightarrow$ **Python**

Each jump looked unreasonable at the time.

Dot 1: Hilbert's 17th Problem (1900)

"Is every non-negative polynomial a sum of squares of rational functions?"

- Posed at the 1900 International Congress
- Answered **yes** by Artin (1927)
- A non-negative polynomial $p(x) \geq 0$ can be certified by decomposing it into $\sum q_i^2$
- **No one was thinking about software**



Dot 2: Stengle's Positivstellensatz (1974)

If a system of polynomial inequalities is **infeasible**,

there exists an **algebraic certificate** proving it

$\Rightarrow$ "Infeasibility has a finite proof in the polynomial ring"

- Generalizes Hilbert–Artin to **semialgebraic sets** (regions defined by polynomial $\geq, =, \leq$)
- If two regions don't overlap, you can *prove* it with a polynomial identity
- Still pure math. No computers, no programs, no control systems.

Key insight we'll use: if safe $\cap$ unsafe $= \varnothing$, a polynomial *fence* must exist.

Dot 3: Parrilo's SOS Programming (2000)

The jump to **control theory**

- **Problem:** control engineers need to prove robots / aircraft / reactors **stay safe**
- Lyapunov stability: find a function $V(x)$ that proves the system never leaves a safe region
- Parrilo: checking "$p(x) \geq 0$" is equivalent to a **semidefinite program** (SDP)
- **Suddenly computable**

$$p(x) = \sum_i q_i(x)^2$$

$$\Updownarrow$$

Solve an SDP for the Gram matrix

Hilbert's 1900 question $\rightarrow$ a **tractable optimization problem** in 2000.

Dot 4: Prajna's Barrier Certificates (HSCC 2004)

The jump from **control theory** to **verification**

Init
$\forall s \in S_0 : \; B(s) \geq \epsilon$

→

Consecution
$B(s) \geq 0 \Rightarrow B(s') \geq 0$

→

Separation
$\forall s \in U : \; B(s) \leq -\epsilon$

- Prajna & Jadbabaie: use barrier functions to prove **safety**, not just stability
- Three obligations. If all hold, unsafe states are **provably unreachable**.
- Originally for continuous dynamical systems (robots, circuits).

💡 We use this **verbatim** — over Python bytecode state.

Dot 5: The 20 Techniques We Stole

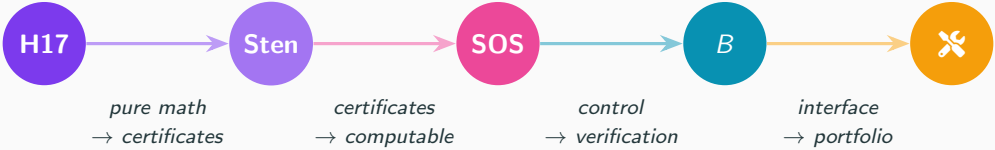
With the barrier **interface** in hand,

every verification technique becomes a **specialized solver**
for one or more of the three obligations.

	CHC/Spacer	Houdini/ICE
DSOS/SDSOS	k-induction	Interpolants
SOS/SDP	IC3/PDR	CEGAR
Init	**Consecution**	**Separation**

Every paper answers one question about B. Composition is **coherent**, not eclectic.

Connecting the Dots



126 years of seemingly unrelated work.

Each dot looked like a different field entirely.
Together they form a **complete Python bug-finding pipeline**.

Why a New Theoretical Basis?

Existing verification methods work. Why didn't we just *use* them?

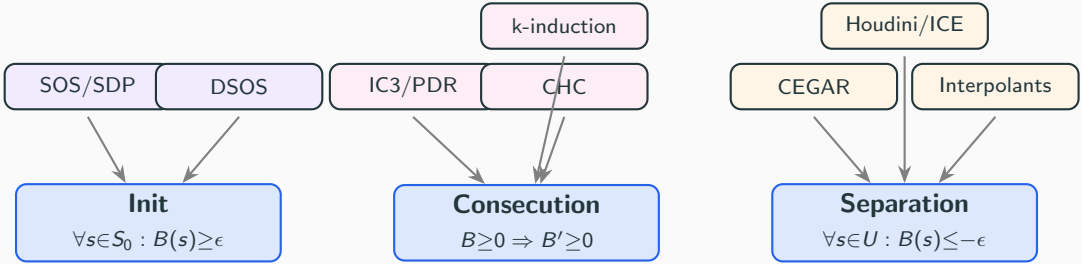
What existing methods give you:

- Abstract interpretation: sound, but **no composition API** across techniques
- Model checking: precise, but **state explosion** for numeric programs
- SMT alone: powerful, but **no structured proof artifact** you can audit
- Each method works in isolation; combining them is ad hoc

What barrier theory adds:

- **Three universal obligations** that any method can discharge
- Methods **compose** because they all serve the same B
- Failure is **informative**: obligation counterexamples route to the next method
- Every decision maps to an **auditable proof object**
- A shared interface turns a paper zoo

The Unifying Interface



Every paper answers one question about B. Composition is structural, not ad hoc.

Why Barriers Beat the Status Quo

Linters

- Fast, no proofs
- Can't reason about paths
- 80–95% false positives
- $\rightarrow$ alert fatigue

Testing/fuzzing

- Finds bugs, never proves absence
- 2^{64} inputs
- Can't close issues as "safe"
- Expensive

Barrier certs

- **Mathematical proof**
- Path-sensitive
- Handles loops
- **Eliminates silently**

On real repos: **99% of candidate bugs proven unreachable.**
Only $\sim$1% survive for triage.

Proof, Not Confidence

	Linter	LLM triage	Barrier cert.
Proves absence	✗	✗	✓
Auditable artifact	✗	∼	✓
Path-sensitive	✗	∼	✓
Handles loops	✗	✗	✓
Scales to CI	✓	∼	✓

A barrier certificate is a **proof object**: template + coefficients + checked obligations + site metadata.

You can audit it. You can replay it. It doesn't hallucinate.

Concrete Example: DIV_ZERO with a Polynomial Barrier

```
def safe_divide(a, b):
    if b != 0:
        return a / b  # safe?
    return 0.0
```

State variable: $x = \mathtt{b}$

Unsafe set: $U = \{x : x = 0\}$

Guard: division reached only when $x \neq 0$

Choose barrier: $B(x) = x - \frac{1}{2}$

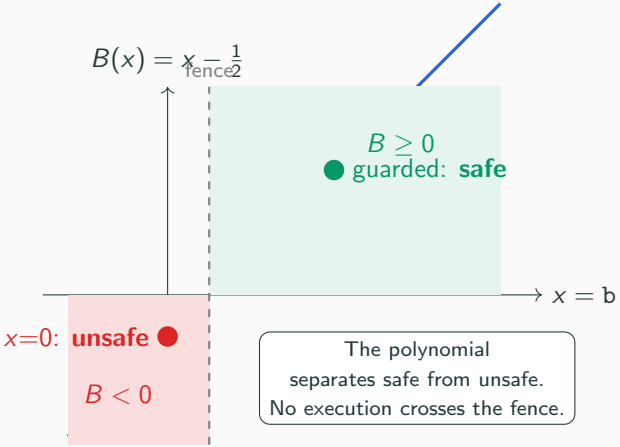
Check three obligations:

1. ✔ **Init:** after guard, $|x| \geq 1$ (integers) $\Rightarrow B(x) \geq \frac{1}{2} > 0$
2. ✔ **Consecution:** x is unchanged on the path to division
3. ✔ **Separation:** $B(0) = -\frac{1}{2} < 0$

$\Rightarrow$ **DIV_ZERO is unreachable.**

Proven by Hilbert $\rightarrow$ Stengle $\rightarrow$ Parrilo $\rightarrow$ Prajna.

The Polynomial Fence, Visually



The polynomial separates safe from unsafe. No execution crosses the fence.

The Portfolio: 20 Papers, One Framework

Layer 6 **Synthesis & Grounding** — Houdini, ICE, SyGuS, DART/concolic #19–20+

↑

Layer 5 **Refinement** — CEGAR, SLAM, interpolants, lazy abstraction #15–18

↑

Layer 4 **Consecution Solvers** — k-induction, IC3/PDR, CHC/Spacer #12–14

↑

Layer 3 **Speed & Scale** — DSOS/SDSOS, sparsity, CBF contracts #9–11

↑

Layer 2 **Barrier Core** — Prajna hybrid, stochastic, SOS safety, SOSTOOLS #5–8

↑

Layer 1 **Foundations** — Stengle, Parrilo SOS, Lasserre, Sparse SOS #1–4

#1 Stengle 1974 — Positivstellensatz

If safe and unsafe regions are truly disjoint,
a polynomial **fence** B **must exist**

The existence guarantee for all barrier certificates

Barrier role: Theoretical bedrock. We *know* a proof exists before we search for one.

How we bent it: Map Python state variables $\rightarrow$ polynomial ring; interpret guards and error conditions as semialgebraic sets

#2 Parrilo 2000 — SOS via SDP

Polynomial positivity $\longleftrightarrow$ Semidefinite Program

"Fence exists" $\rightarrow$ "**solve for its coefficients**"

Barrier role: The computational engine for init and separation constraints.

How we bent it: Parrilo's domain is Lyapunov for ODEs; we substitute the ODE state space with Python's symbolic machine state σ

#3 Lasserre 2001 — Moment/SOS Hierarchy

Degree-d certificate fails? Escalate to $d+2$

Hierarchy **converges** — systematic completeness

Barrier role: Completeness ladder. No degree ceiling stops us permanently.

How we bent it: Originally polynomial *optimization*; we repurpose as *feasibility-escalation* schedule — each level is a portfolio tier, not an optimization step

#4 Sparse SOS (Waki et al., 2006)

Python variables rarely all interact

Decompose the SDP along the **variable correlation graph**

$\Rightarrow$ many small problems instead of one huge one

Barrier role: Scalability for real functions with 10+ locals.

How we bent it: Original graph from polynomial support; we build from Python *data-flow* — co-appearance in branch predicates or arithmetic

#5 Prajna & Jadbabaie, HSCC 2004 — Barrier Certificates

The **three-obligation API** we use verbatim:

$$\underset{\text{init}}{\forall s \in S_0 : B(s) \geq \epsilon} \qquad \underset{\text{consec}}{B(s) \geq 0 \Rightarrow B(s') \geq 0} \qquad \underset{\text{sep}}{\forall s \in U : B(s) \leq -\epsilon}$$

Barrier role: *The* paradigm. Every other paper serves these three conditions.

How we bent it: Replace Lie-derivative consecution ($\dot{B} \geq 0$ along ODE flow) with symbolic one-step $B(\sigma') \geq 0$ under the discrete transition relation T

#6 Prajna et al., TAC 2007 — Stochastic Barriers

Consecution as **expectation inequality**:

$$\mathbb{E}[B(s') \mid s] \geq 0 \text{ when } B(s) \geq 0$$

Barrier role: Extends barriers to non-deterministic/probabilistic transitions.

How we bent it: Replace Itô calculus over SDEs with Python probabilistic branching; expectation over path-condition probability mass

#7 SOS Safety Checker #8 SOSTOOLS Framework

#7: SOS Safety

- "Can *any* B separate these regions?"
- Fast feasibility pre-check
- Skip expensive synthesis if no fence can exist

#8: SOSTOOLS Engineering

- Configurable degree, variables
- Parametric template interface
- Production-grade solver plumbing

Role: Make SOS synthesis *practical* and configurable for CI throughput.

#9 Ahmadi & Majumdar, 2019 — DSOS/SDSOS

Same fence, **cheaper paint**

DSOS $\rightarrow$ LP SDSOS $\rightarrow$ SOCP SOS $\rightarrow$ SDP

50–200$\times$ faster. The fence is still **sound**.

Barrier role: Speed tier. DSOS runs first; escalate to SOS only on failure.

How we bent it: Originally robotics trajectory optimization; we discard the control objective and use it purely for certificate-existence checking

#10 CBF Survey (Ames et al., 2019)

Continuous consecution: $\dot{B} \geq -\alpha(B)$

Slope bound $\rightarrow$ **contract over-approximation** for unknown calls

Barrier role: Motivates how to handle library calls we can't analyze.

How we bent it: Discretize the class-$\mathcal{K}$ slope bound into a per-call contract: $B(\sigma') \geq B(\sigma) - \alpha(B(\sigma))$ for black-box Python functions

#11 Correlative Sparsity in Practice

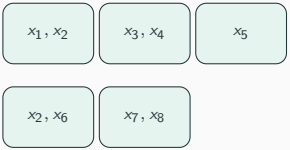
Dense SDP

All n variables

coupled together

$O(n^{2d})$ time

$\Rightarrow$



Sparse: many small SDPs

Variables in different branches rarely interact.

Sparsity makes barrier synthesis tractable for real Python functions.

#12 k-Induction (Sheeran et al., FMCAD 2000)

Does B stay ≥ 0 for k steps, then **forever**?

Bounded base + inductive step $\Rightarrow$ consecution proof

How we bent it: Propositional SAT over hardware bit-vectors $\rightarrow$ SMT over integer/real program variables. Same base/step structure.

#13 IC3 (Bradley, VMCAI 2011)

Build B **implicitly** as a conjunction of learned clauses

Each blocked cube = one constraint maintaining the fence

How we bent it: Boolean SAT over hardware registers $\rightarrow$ LIA predicates over Python state. Z3 model-based quantifier instantiation replaces SAT generalization.

#14 CHC/Spacer (Gurfinkel et al., CAV 2015)

Software safety as **Constrained Horn Clauses**

CHC solving *is* consecution, written **interprocedurally**

How we bent it: SeaHorn targets C/LLVM; we re-encode for Python's transition system. Dynamic dispatch requires per-callsite CHC heads.

#15 CEGAR (Clarke et al., CAV 2000)

Failed obligation = **gap in the fence**

Add monomials to template → re-solve
Finite refinement until fixpoint or budget

How we bent it: Original "abstraction" is a boolean program; ours is the polynomial template degree and monomial basis

#16 SLAM (Ball et al., PLDI 2001)

Predicate abstraction + CEGAR at **Windows driver scale**

Automated fence-gap patching works in **real software**

How we bent it: We take the *architectural* lesson and substitute a polynomial monomial selector for SLAM's boolean predicate engine

#17 Craig Interpolants (McMillan, CAV 2003)

From the UNSAT proof, extract the **minimal new monomial** that closes the gap in the fence

Tightest possible refinement

How we bent it: Original propositional/linear interpolants; we apply SMT interpolation over non-linear arithmetic

#18 Lazy Abstraction (Henzinger et al., POPL 2004)

Don't rebuild the entire B when one part fails

Surgically patch only the template branch that broke

How we bent it: "Subtree" in BLAST is a CFG node set; in ours it's the monomial support region of B around the failing check

#19 Houdini (Flanagan & Leino, 2001) + ICE (Garg et al., 2014)

Houdini
Start with many candidate coefficient configs.
Prune by counterexample.
Fixpoint = **widest valid** B.

ICE Learning
Learn barrier from positive, negative, implication examples.
Program's own behavior shapes B.

How we bent them: Houdini proposes boolean annotations for ESC/Java $\rightarrow$ we generate polynomial *coefficient* candidates. ICE targets general first-order invariants $\rightarrow$ we restrict to bounded-degree polynomials.

#20 SyGuS (Alur et al., 2013) + DART/Concolic (Godefroid et al., 2005)

SyGuS
Template family = grammar.

Three obligation checks = oracle.

Formalizes our coefficient search.

DART/Concolic
Walk **real executions** to confirm

$B \geq 0$ on live paths.

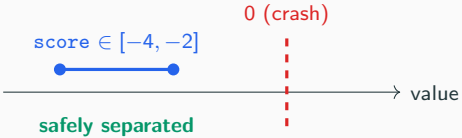
Ground truth for the certificate.

How we bent them: SyGuS is generic (strings, bits, etc.) $\rightarrow$ we instantiate with a monomial grammar. DART's goal is *falsification* $\rightarrow$ we *invert* it to *confirmation*.

Contracts: How PyTorch Meets Barriers

Problem: the analyzer hits `torch.cosine_similarity()` — what is the output?

```
sim = torch.cosine_similarity(x, y)
score = sim - 3
result = 1.0 / score  # DIV_ZERO?
```



Without contract: `sim` is unknown $\rightarrow$ FP.

With contract: `sim` $\in [-1, 1]$

$\Rightarrow$ `score` $\in [-4, -2]$

$\Rightarrow 0 \notin [-4, -2]$

$\Rightarrow$ **Division is provably safe.**

Contracts Are Barrier Constraints on Library Outputs

Contract
`cos_sim` $\in [-1, 1]$

→

Interval algebra
`score` $\in [-4, -2]$

→

Barrier check
$0 \notin [-4, -2]$ ✔

Soundness: $\text{Sem}_f \subseteq R_f$ (contract over-approximates actual behavior).

Consecution with contracts:

$$B(s) \geq 0 \;\wedge\; T_{\text{prog}}(s, s') \;\wedge\; C_{\text{library}}(s, s') \;\Rightarrow\; B(s') \geq 0$$

Better contracts → tighter intervals → more bugs proven unreachable → less noise.

PyTorch Contract Examples

Function	Output bounds	Precondition	Barrier effect
`torch.rand(*size)`	$[0, 1)$	—	excludes < 0 from downstrea
`torch.zeros(*size)`	$= 0$	—	known constant
`torch.eye(n)`	$[0, 1]$	—	bounded outputs
`torch.arange(s,e,st)`	$[s, e-st]$	$st \neq 0$	**flags DIV_ZERO if** $st = 0$
`cos_similarity`	$[-1, 1]$	—	tight interval propagation
`view(*shape)`	shape-preserving	$\prod = \text{numel}$	**shape safety**

1,500+ lines of PyTorch contracts. Each one is a barrier constraint on library outputs that composes through interval algebra.

What Actually Shipped

```
pip install a3-python
```

Static stage (bulk)

- Bytecode analysis + Z3
- Barrier certificate proofs
- 67 bug types
- **99% eliminated**

Agentic stage (residue)

- Multi-turn LLM tool-use
- Reads files, searches guards
- Zero API keys in CI
- Handles the $\sim$1%

Proof artifacts are upstream. LLM judgment is downstream.

The Meta-Lesson

1. **Hilbert** asked if non-negative polynomials are sums of squares (1900)
2. **Stengle** proved polynomial infeasibility has algebraic certificates (1974)
3. **Parrilo** made this computable with SDPs — for *control theory* (2000)
4. **Prajna** wrote three barrier obligations — for *verification* (2004)
5. **We** stole 20 techniques, bent them into those three obligations, and applied them to *Python bytecode* (2024–2026)

The dots were always there. We just drew the line.

Questions?

```
pip install a3-python
a3 scan .  --triage
```

From Hilbert's 17th Problem to your CI pipeline.